

# Personal Workout App & Coaching System

## Design & Technical Specification

Version 2.0

---

### 1. Document Control

| Field          | Detail                                                                    |
|----------------|---------------------------------------------------------------------------|
| Document Title | Personal Workout App & Coaching System — Design & Technical Specification |
| Version        | 2.0                                                                       |
| Status         | Final Draft                                                               |
| Date           | June 2026                                                                 |
| Author         | Engineering Team                                                          |

#### 1.1 Change Log

| Version | Description                                                                                                                                                                                                                                                                                                                                                                                                 |
|---------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1.0     | Initial specification. Covered weekly training structure, eight warm-up/cool-down protocol types, Full Body A/B session designs, core data model (exercises, workout_sessions, session_sets), six FastAPI endpoints, four Flutter screens, and an eight-phase implementation roadmap.                                                                                                                       |
| 2.0     | Added Table of Contents. Incorporated six design improvements: multi-profile support, local notification/reminder system, adaptive LLM feedback loop, proactive deload scheduling, cardio-aware strength recommendations, and RPE trend alerting. Added two architecture notes: DuckDB concurrency constraints and backup/export strategy. Expanded example logged session to show computed backend output. |

#### 1.2 Purpose

This document specifies the design, data model, API contracts, and implementation roadmap for a local-first personal workout tracking and coaching application. The system targets a home-based athlete who wants evidence-based strength training, progressive overload tracking, and rule-driven coaching suggestions, optionally enhanced by a locally hosted large language model.

#### 1.3 Intended Audience

This specification is intended for the sole developer or small team building the application, future contributors extending the codebase, and the end user who wishes to understand the data structures and coaching logic.

---

## **2. Table of Contents**

- 1. Document Control**
- 2. Table of Contents**
- 3. Overview and Goals**
- 4. Weekly Training Structure**
- 5. Warm-up and Cool-down Protocols**
  - 5.1 Protocol Types
  - 5.2 Protocol Quick Reference
- 6. Session Design**
  - 6.1 Full Body A
  - 6.2 Full Body B
  - 6.3 Progression and Volume Rules
- 7. Multi-Profile Support**
- 8. Notification and Reminder System**
- 9. Adaptive LLM Coaching**
- 10. Deload Scheduling**
- 11. Cardio-Aware Strength Recommendations**
- 12. RPE Trend Alerting**
- 13. System Architecture**
- 14. Data Model**
  - 14.1 Core Tables
  - 14.2 Extended Tables (v2.0)
  - 14.3 Derived Analytics Views
- 15. Backend API**
  - 15.1 Suggestion Rules
- 16. Flutter Frontend Structure**
- 17. Goal-Based Adaptive Planning**
- 18. Local LLM Integration**
- 19. Architecture Constraints**
- 20. Data Import and External Integration**

- 21. Implementation Roadmap
- 22. Example Logged Session
- 23. Definitions and Abbreviations

### 3. Overview and Goals

The application is a pure local stack: a Flutter frontend communicates with a Python FastAPI backend and a DuckDB database running on the same machine. All personal workout data is stored locally. An optional Ollama-hosted LLM adds natural-language coaching explanations without any cloud dependency.

#### 3.1 Fitness Context

The system supports a user training primarily at home using bodyweight, a loaded backpack, resistance bands, and dumbbells. Evidence-based guidelines for hypertrophy and strength suggest three to four full-body sessions per week with progressive overload and four to twelve hard sets per major muscle group. The application enforces these principles through its volume tracking and suggestion logic.

#### 3.2 Design Principles

- **Local-first:** all data lives on the user's machine; no cloud account is required.
- **Evidence-based:** volume landmarks (MEV, MAV, MRV) and progression rules are grounded in peer-reviewed exercise science.
- **Deterministic core:** coaching suggestions are generated by rule-based logic; the LLM layer only produces explanatory and conversational prose.
- **Adaptive:** user feedback on suggestions adjusts rule weights over time, making coaching progressively more personalised.
- **Recovery-aware:** cardio load and weekly fatigue signals are factored into strength recommendations.
- **Extensible:** import pipelines accept historic CSV exports and wearable FIT/GPX files without changing the core schema.

### 4. Weekly Training Structure

Three full-body strength sessions per week form the core programme, complemented by cardiovascular and mobility work. The schedule below represents the default seven-day cycle; the user may compress or extend it based on availability.

| Day   | Type        | Focus                 |
|-------|-------------|-----------------------|
| Day 1 | Full Body A | Push + squat emphasis |

| Day   | Type                  | Focus                         |
|-------|-----------------------|-------------------------------|
| Day 2 | Cardio / Mobility     | Cycling, running, stretching  |
| Day 3 | Full Body B           | Pull + hinge emphasis         |
| Day 4 | Rest or light cardio  | Recovery                      |
| Day 5 | Full Body A variant   | Slightly heavier or more sets |
| Day 6 | Optional conditioning | Mobility or light cardio      |
| Day 7 | Rest                  | Full recovery                 |

## 5. Warm-up and Cool-down Protocols

Proper warm-up and cool-down are non-negotiable components of every session. Dynamic warm-ups prepare joints and muscles better than static stretching before lifting. Cool-down stretching and breathwork accelerate recovery and reduce delayed-onset muscle soreness. The application surfaces the correct protocol sequence based on the selected workout template.

| Activity / Session             | Recommended Warm-up Types                                         | Recommended Cool-down Types                          |
|--------------------------------|-------------------------------------------------------------------|------------------------------------------------------|
| Full Body A (push + squat)     | Dynamic full-body + Lower-body focus + Glute activation           | Lower-body static + Recovery walk + Breathwork       |
| Full Body B (pull + hinge)     | Dynamic full-body + Upper-body focus + Posterior chain activation | Upper-body static + PNF hamstring/lat + Breathwork   |
| Running / Cardio               | Sport-specific running warm-up + Hip mobility                     | Light walk + Lower-body static + Diaphragmatic reset |
| Cycling                        | Sport-specific cycling warm-up + Hip flexor / ankle mobility      | Quad/hip flexor static + Recovery walk               |
| Strength deload / Mobility day | Foam rolling + Thoracic mobility                                  | Full-body static + Savasana                          |

### 5.1 Protocol Types

### **Type 1 — Dynamic Warm-up**

**Duration:** 7–10 min **Best paired with:** Every session

Active, movement-based exercises raising core temperature and preparing joints. Full-body routine: march in place, arm circles, leg swings, bodyweight squat, hip circles, inchworm to push-up, world's greatest stretch. Lower-body add-on for Full Body A; upper-body add-on for Full Body B.

### **Type 2 — Foam Rolling (SMR)**

**Duration:** 4–8 min **Best paired with:** Before dynamic warm-up / post-session

Self-myofascial release applied before the dynamic warm-up or as part of the cool-down. At least 120 seconds per area is required for meaningful blood lactate clearance. Key areas: thoracic spine, quads, IT band, lats, calves, hamstrings, glutes.

### **Type 3 — Mobility / Joint Prep**

**Duration:** 5–8 min **Best paired with:** Before leg or press work

Targeted joint-preparation inserted between foam rolling and the main dynamic warm-up on days with known tightness. Hip and ankle routine for leg-dominant sessions; thoracic spine and shoulder routine for push/pull sessions.

### **Type 4 — Muscle Activation**

**Duration:** 4–5 min **Best paired with:** Before Full Body B

Brief targeted patterns that activate key muscles before the main session: glute and posterior chain activation, and core and shoulder-girdle activation for pressing or plank work.

### **Type 5 — Sport-Specific Warm-ups**

**Duration:** 8–10 min **Best paired with:** Before runs or rides

Activity-specific movement patterns mirroring the biomechanics of the sport: running warm-up, cycling warm-up, or bodyweight/HIIT warm-up.

### **Type 6 — Static Cool-down Stretching**

**Duration:** 8–10 min **Best paired with:** After every strength or cardio session

Static stretches held 30–60 s post-workout when muscles are warm. Lower-body routine after Full Body A or running; upper-body and full-body routine after Full Body B.

### Type 7 — PNF Stretching

**Duration:** 8–12 min **Best paired with:** After Full Body B or mobility day

Proprioceptive Neuromuscular Facilitation for advanced flexibility gains. Three variants are supported: Hold-Relax, Contract-Relax, and Contract-Relax-Antagonist-Contract (CRAC).

### Type 8 — Breathwork & Nervous System Reset

**Duration:** 2–5 min **Best paired with:** After every session

Deliberately shifts the nervous system from sympathetic to parasympathetic, speeding heart-rate recovery and reducing cortisol. Techniques: diaphragmatic breathing (4-count inhale, 6–8-count exhale), box breathing, and progressive muscle relaxation.

## 5.2 Protocol Quick Reference

| Protocol Type            | Phase     | Duration | Best Paired With                  |
|--------------------------|-----------|----------|-----------------------------------|
| Dynamic full-body        | Warm-up   | 7–10 min | Every session                     |
| Foam rolling (pre)       | Warm-up   | 4–6 min  | Before dynamic warm-up            |
| Mobility / joint prep    | Warm-up   | 5–8 min  | Before leg or press work          |
| Muscle activation        | Warm-up   | 4–5 min  | Before Full Body B                |
| Sport-specific (running) | Warm-up   | 8–10 min | Before runs                       |
| Sport-specific (cycling) | Warm-up   | 8–10 min | Before rides                      |
| Recovery walk            | Cool-down | 5–8 min  | After any session                 |
| Lower-body static        | Cool-down | 8–10 min | After Full Body A or running      |
| Upper-body static        | Cool-down | 8–10 min | After Full Body B                 |
| Foam rolling (post)      | Cool-down | 5–8 min  | After any session                 |
| PNF (Hold-Relax)         | Cool-down | 8–12 min | After Full Body B or mobility day |
| Diaphragmatic breathing  | Cool-down | 2–3 min  | After every session               |
| Box breathing            | Cool-down | 2–4 min  | After high-RPE sessions           |
| Savasana / PMR           | Cool-down | 3–5 min  | After long sessions               |

## 6. Session Design

## 6.1 Full Body A

Full Body A emphasises squats, horizontal pushing, glute work, and core. Target muscle groups: quads, glutes/hamstrings, chest, shoulders, upper back, calves, and core.

| Exercise                  | Default Variation                    | Sets | Rep Range           |
|---------------------------|--------------------------------------|------|---------------------|
| Squat                     | Bodyweight → backpack goblet squat   | 2–3  | 8–15                |
| Push-up                   | Incline, standard, or decline        | 2–3  | 6–12                |
| Hip thrust / Glute bridge | Bodyweight → backpack-loaded         | 2–3  | 10–15               |
| Horizontal row            | Inverted row, band row, dumbbell row | 2–3  | 8–12                |
| Calf raise                | Bodyweight                           | 2    | 12–20               |
| Core                      | Plank or dead bug                    | 2–3  | 20–40 s / 8–12 reps |

## 6.2 Full Body B

Full Body B emphasises unilateral lower-body work, hip hinge, pulling, and shoulder pressing, complementing Full Body A.

| Exercise                   | Default Variation                      | Sets | Rep Range      |
|----------------------------|----------------------------------------|------|----------------|
| Unilateral squat           | Bulgarian split squat or reverse lunge | 2–3  | 8–12 / leg     |
| Hip hinge                  | Romanian deadlift or good morning      | 2–3  | 8–12           |
| Horizontal / vertical pull | Inverted row, band row, dumbbell row   | 2–3  | 8–12           |
| Overhead push              | Pike push-up or shoulder press         | 2–3  | 6–12           |
| Single-leg glute bridge    | Unilateral hip thrust                  | 2–3  | 10–15 / leg    |
| Core anti-rotation         | Side plank or shoulder taps            | 2–3  | 15–30 s / side |

## 6.3 Progression and Volume Rules

- **Intensity:** finish each set with approximately 1–3 reps in reserve (RIR), corresponding to an RPE of 7–9.
- **Weekly volume:** target 6–12 hard sets per major muscle group per week (hard set =  $\text{RIR} \leq 3$  or  $\text{RPE} \geq 7$ ).
- **Progression:** once the top of the rep range is achievable across all sets with good form, add load, use a harder variation, or add one set.

- **Deloads:** handled by the proactive deload scheduling module described in Section 10.
- 

## 7. Multi-Profile Support

Version 1.0 stored all data in a single global context, meaning any user of the machine would read and write to the same session history and coaching state. Version 2.0 introduces a lightweight local profile switcher that keeps each user's data fully isolated without requiring authentication credentials.

### 7.1 Design Approach

Each profile is a named context stored as a separate DuckDB file under a `profiles/` directory within the application data folder. The active profile is persisted in a local `config.json` file. On launch, the Flutter app reads `config.json` and connects the backend to the corresponding database file. A profile management screen allows the user to create, rename, switch, and delete profiles.

### 7.2 Schema Changes

| Addition                             | Type      | Purpose                                                                                    |
|--------------------------------------|-----------|--------------------------------------------------------------------------------------------|
| <code>profiles/</code>               | Directory | One <code>.duckdb</code> file per named profile (e.g. <code>profiles/jamie.duckdb</code> ) |
| <code>config.json</code>             | File      | Stores <code>active_profile</code> name and app-level preferences                          |
| <code>GET /profiles</code>           | Endpoint  | List available profiles                                                                    |
| <code>POST /profiles</code>          | Endpoint  | Create a new profile (creates a new <code>.duckdb</code> and seeds it)                     |
| <code>PUT /profiles/active</code>    | Endpoint  | Switch the active profile; backend reconnects to the new <code>.duckdb</code>              |
| <code>DELETE /profiles/{name}</code> | Endpoint  | Delete a profile and its database file after confirmation                                  |

### 7.3 Flutter Integration

A profile-switcher screen is accessible from the settings menu. It lists existing profiles, shows which is active, and provides buttons to add or remove profiles. Switching profiles triggers a `PUT /profiles/active` call and refreshes all screens. No re-launch is required.

---

## 8. Notification and Reminder System

Version 1.0 had no mechanism to prompt the user to train. Version 2.0 adds a local notification layer using the `flutter_local_notifications` package, which schedules OS-level notifications without a server or internet connection.



## 8.1 Notification Types

| Notification       | Trigger                                            | Default Timing                          |
|--------------------|----------------------------------------------------|-----------------------------------------|
| Session reminder   | User-configured training days (from user_settings) | 30 min before preferred training window |
| Streak alert       | No strength session logged in last 4 days          | 08:00 on day 5                          |
| Deload reminder    | Proactive deload week flagged (see Section 10)     | Morning of first day of deload week     |
| RPE alert          | RPE trend alert fired (see Section 12)             | Next morning after the flagging session |
| Post-session nudge | Session submitted but no cool-down logged          | 5 min after session submission          |

## 8.2 Implementation Notes

- flutter\_local\_notifications handles Android and iOS notification channels. On desktop (Linux/macOS/Windows) the equivalent is flutter\_local\_notifications with the appropriate platform plugin.
- Notification schedules are stored in a notifications table in config.json (not in the profile .duckdb) so they persist across profile switches.
- The user can enable, disable, or retime each notification type from the Settings screen.
- No push infrastructure is needed — all scheduling is handled by the OS scheduler via the plugin.

# 9. Adaptive LLM Coaching

Version 1.0's LLM integration was one-directional: the model explained pre-computed rule-based suggestions but had no awareness of whether those suggestions were useful. Version 2.0 adds a feedback loop that adjusts rule weights based on user responses, making coaching progressively more personalised over time.

## 9.1 Feedback Mechanism

Each suggestion card in the Coach screen displays a thumbs-up and thumbs-down button. Tapping either logs a suggestion\_feedback record linking the suggestion type and outcome to the current week's training context. After five or more feedback events for a given rule, the backend adjusts that rule's confidence weight.

## 9.2 Rule Weight Adjustment

| Scenario                                              | Adjustment                                                                          |
|-------------------------------------------------------|-------------------------------------------------------------------------------------|
| User consistently thumbs-down 'add a set' suggestions | Raise the hard_sets threshold for that muscle group by 1 before triggering the rule |

| Scenario                                                | Adjustment                                                              |
|---------------------------------------------------------|-------------------------------------------------------------------------|
| User consistently thumbs-up 'load increase' suggestions | Lower the staleness threshold from 3 weeks to 2 weeks for that exercise |
| Mixed feedback (< 60% agreement)                        | No adjustment; rule fires at default thresholds                         |
| Feedback on LLM explanation quality                     | Stored separately; used to refine prompt templates, not rule weights    |

### 9.3 Schema Addition

A `suggestion_feedback` table is added to the profile database:

| Column                   | Type      | Notes                                                    |
|--------------------------|-----------|----------------------------------------------------------|
| <code>feedback_id</code> | INTEGER   | Primary key                                              |
| <code>session_id</code>  | INTEGER   | The session during which the suggestion was shown        |
| <code>rule_key</code>    | TEXT      | Identifier of the rule that fired (e.g. 'add_set_quads') |
| <code>rating</code>      | INTEGER   | 1 = thumbs-up, -1 = thumbs-down                          |
| <code>logged_at</code>   | TIMESTAMP | UTC timestamp of the rating                              |

## 10. Deload Scheduling

Version 1.0 mentioned deloads only reactively — when performance dropped. Version 2.0 adds a proactive deload scheduler that plans a reduced-volume week at a fixed interval regardless of whether fatigue has been detected, in line with standard periodisation practice.

### 10.1 Scheduling Logic

- **Default cadence:** one deload week every four to six training weeks. The exact interval is configurable in `user_settings` (`deload_interval_weeks`, default 5).
- **Reactive override:** if two consecutive weeks of declining performance are detected before the scheduled deload, the deload is brought forward to the following week.
- **Deload definition:** all working sets reduced by 25–30%; load kept the same; RIR target relaxed to 4–5. The template sequence (Full Body A/B) continues unchanged.
- **Deload detection:** the backend marks a week as a deload week in a `training_blocks` table. The suggestion engine suppresses volume-increase recommendations during deload weeks.

### 10.2 Schema Addition

| Column          | Type    | Notes                                             |
|-----------------|---------|---------------------------------------------------|
| block_id        | INTEGER | Primary key                                       |
| week_start      | DATE    | Monday of the training week                       |
| block_type      | TEXT    | training   deload                                 |
| trigger         | TEXT    | scheduled   reactive                              |
| deload_interval | INTEGER | Configured interval at time of scheduling (weeks) |

## 11. Cardio-Aware Strength Recommendations

Version 1.0 imported cardio sessions into a cardio\_sessions table but did not use that data when generating strength recommendations. Version 2.0 integrates cardio load into the suggestion engine so that high-volume running or cycling weeks reduce the recommended leg volume for subsequent strength sessions.

### 11.1 Cardio Load Calculation

Each cardio session is assigned a Cardio Stress Score (CSS) based on duration and average heart rate. The CSS is used to compute a seven-day rolling cardio load for the lower-body muscle groups (quads, hamstrings, glutes, calves) that running and cycling primarily stress.

| Activity | Primary Muscle Groups Stressed    | CSS Contribution                                                                                 |
|----------|-----------------------------------|--------------------------------------------------------------------------------------------------|
| Running  | Quads, hamstrings, glutes, calves | $\text{duration\_min} \times \text{HR\_factor}$ (HR_factor = $\text{avg\_hr} / \text{max\_hr}$ ) |
| Cycling  | Quads, glutes, hip flexors        | $\text{duration\_min} \times \text{HR\_factor} \times 0.8$ (lower eccentric load)                |
| Rowing   | Back, glutes, hamstrings          | $\text{duration\_min} \times \text{HR\_factor} \times 0.7$                                       |
| Swimming | Upper back, shoulders             | $\text{duration\_min} \times \text{HR\_factor} \times 0.6$                                       |

### 11.2 Recommendation Adjustments

- If the seven-day rolling CSS for lower-body groups exceeds a moderate threshold (configurable; default 120 CSS-minutes), the engine reduces recommended hard sets for quads and hamstrings by one set for the next Full Body session.
- If the CSS exceeds a high threshold (default 200 CSS-minutes), leg volume is reduced by two sets and a note is added to the suggestion explaining the adjustment.
- Upper-body strength recommendations are unaffected by running and cycling load.
- The CSS thresholds are stored in user\_settings and can be adjusted from the Settings screen.

## 12. RPE Trend Alerting

Version 1.0 flagged fatigue only after volume thresholds were crossed. Version 2.0 adds an earlier warning: if the user's RPE on the same exercise at the same or lower load rises across three or more consecutive sessions, the system generates a fatigue alert before volume-based thresholds are triggered.

### 12.1 Detection Logic

| Condition                                                      | Alert Level | Action                                                                                          |
|----------------------------------------------------------------|-------------|-------------------------------------------------------------------------------------------------|
| RPE rises by 1 point over 2 consecutive sessions at same load  | Advisory    | Add an advisory note to the suggestion card for that exercise                                   |
| RPE rises by 1+ point over 3 consecutive sessions at same load | Warning     | Suggest a one-session technique check or load reduction; notify via Section 8 RPE alert         |
| RPE rises by 2+ points over 3 sessions                         | High        | Recommend immediate deload for that muscle group; trigger reactive deload logic from Section 10 |

### 12.2 Implementation Notes

- RPE trend detection runs as part of the GET /suggestions/next pipeline on each call, reading the last eight session\_sets records for each exercise.
- Only sessions where the same exercise was performed at the same or lower weight are included in the trend window — a load increase that causes higher RPE is expected and excluded.
- Alert state is stored in the rpe\_alerts table and cleared when RPE normalises (drops below the initial baseline) or after a deload week.

## 13. System Architecture

The application uses a local-first architecture: the Flutter frontend communicates with a Python FastAPI backend and a per-profile DuckDB database file, all running on the same machine. No network connection is required for normal operation. See Section 19 for concurrency constraints and backup considerations.

| Component              | Technology                 | Responsibility                                                                                          |
|------------------------|----------------------------|---------------------------------------------------------------------------------------------------------|
| Frontend               | Flutter (Dart)             | UI for logging workouts, viewing analytics, managing profiles, and displaying coaching suggestions      |
| Backend                | Python / FastAPI / Uvicorn | REST API for session ingestion, analytics computation, coaching rule evaluation, and profile management |
| Database (per profile) | DuckDB (profiles/*.duckdb) | Relational storage for exercises, sessions, sets, protocol definitions, feedback, and block records     |

| Component            | Technology              | Responsibility                                                                                                                           |
|----------------------|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| Config store         | config.json             | Active profile name, notification schedules, and app-level preferences                                                                   |
| LLM layer (optional) | Ollama (llama3 / gemma) | Natural-language explanations for coaching suggestions via local REST API at <a href="http://localhost:11434">http://localhost:11434</a> |

## 14. Data Model

All tables reside in the active profile's DuckDB file. Application-level configuration is stored separately in config.json. The schema is initialised from SQL migration scripts versioned in the repository.

### 14.1 Core Tables (v1.0)

#### exercises

| Column               | Type    | Notes                                           |
|----------------------|---------|-------------------------------------------------|
| exercise_id          | INTEGER | Primary key                                     |
| name                 | TEXT    | Display name (e.g. 'Goblet squat')              |
| category             | TEXT    | squat   hinge   push   pull   core              |
| primary_muscle_group | TEXT    | quads   glutes   chest   back   shoulders   ... |

#### workout\_sessions

| Column       | Type           | Notes                                          |
|--------------|----------------|------------------------------------------------|
| session_id   | INTEGER / UUID | Primary key                                    |
| session_date | DATE           | Date the session was performed                 |
| template     | TEXT           | FullBodyA   FullBodyB   Run   Cycle   Mobility |
| notes        | TEXT           | Optional free-text session notes               |

#### session\_sets

| Column     | Type           | Notes                          |
|------------|----------------|--------------------------------|
| set_id     | INTEGER / UUID | Primary key                    |
| session_id | INTEGER / UUID | Foreign key → workout_sessions |

| Column         | Type    | Notes                                |
|----------------|---------|--------------------------------------|
| exercise_id    | INTEGER | Foreign key → exercises              |
| set_number     | INTEGER | Ordinal position within the session  |
| reps_completed | INTEGER | Actual reps performed                |
| weight_kg      | REAL    | Load in kilograms (0 for bodyweight) |
| rir            | INTEGER | Reps in reserve at end of set        |
| rpe            | INTEGER | Rate of perceived exertion (1–10)    |

#### warmup\_cooldown\_protocols

| Column            | Type    | Notes                                           |
|-------------------|---------|-------------------------------------------------|
| protocol_id       | INTEGER | Primary key                                     |
| protocol_type     | TEXT    | e.g. 'Dynamic full-body', 'PNF Hold-Relax'      |
| phase             | TEXT    | warmup   cooldown                               |
| duration_min_low  | INTEGER | Minimum recommended duration (minutes)          |
| duration_min_high | INTEGER | Maximum recommended duration (minutes)          |
| session_tags      | TEXT[]  | Templates this protocol applies to              |
| steps             | JSONB   | Ordered list of exercise steps with timing cues |

#### cardio\_sessions

| Column       | Type    | Notes                                     |
|--------------|---------|-------------------------------------------|
| cardio_id    | INTEGER | Primary key                               |
| session_date | DATE    | Date the cardio session was performed     |
| activity     | TEXT    | Run   Cycle   Row   Swim                  |
| duration_min | INTEGER | Total duration in minutes                 |
| distance_km  | REAL    | Distance covered (optional)               |
| avg_hr       | INTEGER | Average heart rate (optional)             |
| css          | REAL    | Computed Cardio Stress Score (Section 11) |

## 14.2 Extended Tables (v2.0)

### suggestion\_feedback

| Column      | Type      | Notes                                         |
|-------------|-----------|-----------------------------------------------|
| feedback_id | INTEGER   | Primary key                                   |
| session_id  | INTEGER   | Session during which the suggestion was shown |
| rule_key    | TEXT      | Rule identifier (e.g. 'add_set_quads')        |
| rating      | INTEGER   | 1 = thumbs-up, -1 = thumbs-down               |
| logged_at   | TIMESTAMP | UTC timestamp of the rating                   |

### training\_blocks

| Column          | Type    | Notes                                             |
|-----------------|---------|---------------------------------------------------|
| block_id        | INTEGER | Primary key                                       |
| week_start      | DATE    | Monday of the training week                       |
| block_type      | TEXT    | training   deload                                 |
| trigger         | TEXT    | scheduled   reactive                              |
| deload_interval | INTEGER | Configured interval at time of scheduling (weeks) |

### rpe\_alerts

| Column      | Type    | Notes                                        |
|-------------|---------|----------------------------------------------|
| alert_id    | INTEGER | Primary key                                  |
| exercise_id | INTEGER | Foreign key → exercises                      |
| alert_level | TEXT    | advisory   warning   high                    |
| fired_at    | DATE    | Date the alert was first generated           |
| cleared_at  | DATE    | Date the alert was resolved (NULL if active) |

### rule\_weights

| Column       | Type      | Notes                                              |
|--------------|-----------|----------------------------------------------------|
| rule_key     | TEXT      | Primary key — matches suggestion_feedback.rule_key |
| weight       | REAL      | Multiplier applied to rule threshold (default 1.0) |
| last_updated | TIMESTAMP | UTC timestamp of last feedback-driven adjustment   |

### 14.3 Derived Analytics Views

| View                 | Key Columns                                          | Purpose                                                                                 |
|----------------------|------------------------------------------------------|-----------------------------------------------------------------------------------------|
| weekly_muscle_volume | muscle_group, week, hard_sets, volume_load           | Hard-set count and volume load per muscle group per calendar week                       |
| exercise_progress    | exercise_id, week, avg_reps, est_1rm                 | Trend in estimated one-rep max or average reps at a given load                          |
| weekly_cardio_load   | muscle_group, week, total_css                        | Seven-day rolling Cardio Stress Score per muscle group for cardio-aware recommendations |
| rpe_trend            | exercise_id, session_date, rpe, weight_kg, delta_rpe | Per-exercise RPE change relative to previous session at same or lower load              |

## 15. Backend API

The FastAPI backend exposes the endpoints below. All responses are JSON. Validation is handled by Pydantic models; errors return standard HTTP 4xx/5xx responses with a detail field.

| Method | Path               | Description                                                                                                                                             |
|--------|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| POST   | /sessions          | Insert a workout session and all associated sets into the active profile database                                                                       |
| GET    | /analytics/weekly  | Return primary_muscle_group, hard_sets, and volume_load for the last 7 days                                                                             |
| GET    | /suggestions/next  | Apply rule-based coaching logic (including cardio load and RPE trend checks) to weekly analytics; return suggestions, recommended_template, and summary |
| GET    | /protocols         | Return the warm-up/cool-down protocol sequence for a given template (?template=FullBodyA)                                                               |
| POST   | /coach/explain     | Construct a prompt from recent analytics and pass to Ollama; return a natural-language explanation of a specific suggestion                             |
| POST   | /coach/chat        | Expose a conversational coaching interface backed by the local LLM                                                                                      |
| POST   | /sessions/feedback | Log a thumbs-up or thumbs-down rating for a suggestion; trigger rule-weight adjustment if threshold is met                                              |
| POST   | /import/csv        | Ingest historic strength data from a CSV export                                                                                                         |
| GET    | /profiles          | List available profiles                                                                                                                                 |



| Method | Path             | Description                                                                 |
|--------|------------------|-----------------------------------------------------------------------------|
| POST   | /profiles        | Create a new profile                                                        |
| PUT    | /profiles/active | Switch the active profile                                                   |
| DELETE | /profiles/{name} | Delete a profile after confirmation                                         |
| GET    | /backup/export   | Export the active profile database as a compressed archive (see Section 19) |

## 15.1 Suggestion Rules

The GET /suggestions/next endpoint applies the following rules, checked in order, after loading the weekly\_muscle\_volume, weekly\_cardio\_load, and rpe\_trend views:

- **RPE trend check (Section 12):** evaluate rpe\_trend for all exercises trained this week; generate alerts before volume-based rules run.
- **Cardio load check (Section 11):** if weekly\_cardio\_load for lower-body groups exceeds thresholds, reduce leg hard\_set targets for this suggestion cycle.
- **Deload check (Section 10):** if training\_blocks marks the current week as a deload, suppress all volume-increase rules and return a deload-mode suggestion set.
- **Volume floor rule:**  $\text{hard\_sets} < \text{MEV} \times \text{rule\_weight}$  and  $\text{average RPE} \leq 7 \rightarrow$  suggest adding a set.
- **Volume ceiling rule:**  $\text{hard\_sets} > \text{MRV} \times \text{rule\_weight}$  or high RPE  $\rightarrow$  suggest reducing sets.
- **Staleness rule:** volume load for an exercise unchanged for  $\text{staleness\_threshold} \times \text{rule\_weight}$  weeks at moderate RPE  $\rightarrow$  suggest load increase or variation change.
- **Template balance rule:** if the last two sessions used the same template, recommend the alternate.

## 16. Flutter Frontend Structure

The Flutter app uses a bottom navigation bar with five primary screens. An ApiClient service class centralises all HTTP communication with the FastAPI backend.

| Screen              | Route     | Key Features                                                                                                        |
|---------------------|-----------|---------------------------------------------------------------------------------------------------------------------|
| Home / Today        | /home     | Recommended template card; next warm-up and cool-down summary; deload-week banner if applicable; quick-start button |
| Workout Logger      | /log      | Exercise list; set-logging form with reps, weight, and RIR/RPE fields; session submission to POST /sessions         |
| History / Analytics | /progress | Weekly hard-set bar charts; estimated 1RM trend lines; RPE trend overlays; session history list                     |

| Screen              | Route     | Key Features                                                                                                                          |
|---------------------|-----------|---------------------------------------------------------------------------------------------------------------------------------------|
| Suggestions / Coach | /coach    | Suggestion cards with thumbs-up/down; 'Why?' button per card; 'Ask Coach' chat interface; feedback logged via POST /sessions/feedback |
| Settings            | /settings | Profile switcher; notification preferences; cardio CSS thresholds; deload interval; goal configuration; backup/export trigger         |

## 16.1 Warm-up and Cool-down Screens

Two dedicated screens, activated before and after the main workout logger respectively, step the user through each protocol exercise with timing cues. The correct protocol sequence is fetched from GET /protocols at the start of each session. Activity-type detection routes cardio sessions to sport-specific protocols rather than strength defaults.

## 17. Goal-Based Adaptive Planning

The user\_settings table stores the user's primary training goal, preferred weekly training frequency, equipment constraints, deload interval, and cardio CSS thresholds. The backend uses goal-specific volume landmarks to personalise coaching suggestions.

| Goal                        | Hard Sets / Muscle / Week | Load & Rep Range                      |
|-----------------------------|---------------------------|---------------------------------------|
| Muscle gain                 | 10–20 sets                | Moderate load, 6–20 reps near failure |
| Strength                    | 8–15 sets                 | Heavy load, 3–6 reps                  |
| Endurance / general fitness | 6–10 sets                 | Light-moderate load, 15–25 reps       |

## 18. Local LLM Integration

Ollama runs a local model (llama3 or gemma) and exposes a REST API at <http://localhost:11434>. The FastAPI backend constructs structured prompts from recent analytics and rule-based suggestions. Core coaching logic remains deterministic; the LLM only generates explanatory and conversational responses. Prompt templates are versioned and refined based on explanation quality feedback collected via the Coach screen.

| Endpoint       | LLM Role                                        | Prompt Construction                                                                             |
|----------------|-------------------------------------------------|-------------------------------------------------------------------------------------------------|
| /coach/explain | Explain a specific suggestion in plain language | Injects muscle group volume data, the triggered rule, active rule_weight, and recent RPE trends |

| Endpoint    | LLM Role                             | Prompt Construction                                                                                                     |
|-------------|--------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| /coach/chat | Answer open-ended training questions | Injects full weekly summary, user goal, current block type (training/deload), and cardio load context as system context |

## 19. Architecture Constraints

### 19.1 DuckDB Concurrency

DuckDB supports only one writer at a time per database file. Concurrent write attempts — for example, the Flutter app submitting a session while a background CSV import is running — will result in the second write being rejected with a locking error. The following mitigations are applied in v2.0:

- The FastAPI backend serialises all writes through a single application-level write queue, preventing concurrent access from within the same process.
- The import endpoint POST /import/csv is disabled while a session is in progress (enforced by a session\_in\_progress flag in the app state).
- Background import tasks are deferred to the next idle period using a simple task queue (asyncio.Queue).
- The architecture section of the developer documentation notes this constraint explicitly so future contributors do not introduce background workers that write to DuckDB concurrently.

### 19.2 Backup and Export Strategy

A single .duckdb file with no export path is a data-loss risk. Version 2.0 adds a structured backup strategy:

- **Manual export:** GET /backup/export compresses the active profile's .duckdb file into a timestamped .tar.gz archive and returns it for download. Accessible from the Settings screen.
- **Automatic local backup:** on successful session submission (POST /sessions), the backend copies the .duckdb file to a backups/ directory, retaining the last seven snapshots.
- **CSV export:** GET /backup/csv exports all session\_sets joined with exercises and workout\_sessions as a flat CSV file, providing a format that is readable without DuckDB tooling.
- **Import round-trip:** exported CSVs can be re-imported via POST /import/csv, enabling migration between machines or profile restoration after accidental deletion.

## 20. Data Import and External Integration

Imported data is indistinguishable from live-logged sessions at the query level, ensuring historic data is treated identically by the analytics and suggestion engines.

## 20.1 Strength History

CSV or JSON exports are ingested via a Python ETL script or the POST /import/csv endpoint. Exercise names in the import file are resolved to exercise\_id values through a configurable mapping table, allowing data from third-party apps to be normalised into the local schema.

## 20.2 Cardio and Wearable Data

FIT and GPX exports are imported into the cardio\_sessions table. The Cardio Stress Score is computed on import and stored in the css column, immediately feeding into the cardio-aware strength recommendation logic described in Section 11.

---

# 21. Implementation Roadmap

The project is divided into ten sequential phases. Phases 1–8 cover the original v1.0 scope; Phases 9–10 deliver the v2.0 additions.

## Phase 1 — Project Scaffold and Core Schema

- Create a Git repository with /backend, /frontend, and /docs directories.
- Initialise a Python virtual environment and install FastAPI, Uvicorn, and DuckDB.
- Create the profile directory structure and config.json schema.
- Create the initial profile .duckdb with SQL initialisation scripts for all core tables.
- Seed the exercises table with the Full Body A/B exercise catalogue.

## Phase 2 — Backend API for Logging and Analytics

- Implement POST /sessions, GET /analytics/weekly, and GET /suggestions/next (baseline rules).
- Add the write serialisation queue to prevent concurrent DuckDB access.
- Add input validation, basic error handling, and unit tests for analytics and suggestion logic.
- Verify endpoints with curl or FastAPI's built-in Swagger UI.

## Phase 3 — Flutter Frontend MVP

- Create the Flutter project with bottom navigation: Home, Log, Progress, Coach, Settings.
- Implement an ApiClient service pointing to the FastAPI backend.
- Build the Workout Logger screen and a basic Coach screen.

## **Phase 4 — Warm-up and Cool-down Module**

- Add the `warmup_cooldown_protocols` table and seed with all protocol types.
- Implement `GET /protocols` and build the Warm-up and Cool-down screens in Flutter.
- Add activity-type detection for cardio templates.

## **Phase 5 — Analytics and Adaptive Planning**

- Extend backend analytics to compute MEV/MAV/MRV per muscle group.
- Implement goal configuration storage and the weekly planning function.
- Enhance the Progress screen with charts for hard sets, volume load, and estimated 1RM.

## **Phase 6 — Data Import and External Integration**

- Create the `POST /import/csv` endpoint and the `cardio_sessions` import pipeline.
- Implement the automatic local backup on session submission.
- Implement `GET /backup/export` and `GET /backup/csv`.
- Add an Import screen and a backup trigger to the Settings screen in Flutter.

## **Phase 7 — Local LLM Integration**

- Configure Ollama with llama3 or a small instruct model.
- Implement `POST /coach/explain` and `POST /coach/chat`.
- Update the Coach screen with 'Why?' buttons and 'Ask Coach' chat interface.

## **Phase 8 — Polish, Testing, and Packaging**

- Add robust validation, logging, and environment-based configuration.
- Write integration tests: `log session` → `analytics` → `suggestions` → `LLM explanations`.
- Improve Flutter UI with consistent theming and mobile-optimised layouts.
- Optionally containerise the backend with Docker via `docker-compose`.

## **Phase 9 — v2.0 Feature Set**

- Implement multi-profile support: profile directory, `config.json`, and `/profiles` endpoints.
- Build the profile-switcher screen in Flutter Settings.
- Integrate `flutter_local_notifications` and implement all five notification types.
- Add the `suggestion_feedback` table and `POST /sessions/feedback` endpoint.
- Implement rule-weight adjustment logic triggered by feedback threshold.
- Add the `training_blocks` table and proactive deload scheduling logic.

- Implement reactive deload override when two consecutive declining-performance weeks are detected.
- Add the cardio CSS calculation on import and the weekly\_cardio\_load view.
- Integrate cardio-aware volume adjustments into GET /suggestions/next.
- Add the rpe\_trend view and RPE alert detection to the suggestion pipeline.
- Add the rpe\_alerts table and alert-clearing logic.

## Phase 10 — v2.0 Polish and Documentation

- Update all unit and integration tests to cover new tables, endpoints, and rule paths.
- Add developer documentation noting the DuckDB concurrency constraint and write queue design.
- Update the Settings screen with all new user-configurable parameters.
- Update the Coach screen to display RPE alerts and deload-week banners.
- Final end-to-end test: full session log → cardio import → adaptive suggestion cycle → feedback → rule-weight update.

## 22. Example Logged Session

The following illustrates a fully logged Full Body A session submitted to POST /sessions, followed by the computed output returned by GET /suggestions/next immediately after. This demonstrates the end-to-end flow from data entry to actionable coaching output.

### 22.1 Session Input

| Field     | Value                                                                                    |
|-----------|------------------------------------------------------------------------------------------|
| Date      | 2026-06-27                                                                               |
| Profile   | jamie                                                                                    |
| Template  | Full Body A                                                                              |
| Warm-up   | Dynamic full-body (8 min) + Lower-body focus add-on (5 min) + Foam rolling (pre-session) |
| Cool-down | Recovery walk (5 min) + Lower-body static (9 min) + Diaphragmatic breathing (3 min)      |

  

| Exercise        | Sets × Reps | Load           | RIR / RPE |
|-----------------|-------------|----------------|-----------|
| Goblet squat    | 3 × 10      | 15 kg backpack | RIR 2     |
| Incline push-up | 3 × 12      | Bodyweight     | RIR 1     |
| Hip thrust      | 3 × 15      | 20 kg backpack | RIR 2     |

| Exercise     | Sets × Reps | Load       | RIR / RPE |
|--------------|-------------|------------|-----------|
| Inverted row | 3 × 10      | Bodyweight | RIR 3     |
| Calf raise   | 2 × 20      | Bodyweight | RIR 1     |
| Plank        | 3 × 30 s    | Bodyweight | RPE 8     |

## 22.2 Computed Backend Output (GET /suggestions/next)

The backend processes the submitted session against four weeks of history and returns the following response:

| Output Field           | Value                                               |
|------------------------|-----------------------------------------------------|
| recommended_template   | Full Body B                                         |
| block_type             | training (week 3 of 5 before next scheduled deload) |
| rpe_alerts             | None                                                |
| cardio_load_adjustment | None (no cardio sessions in last 7 days)            |

| Muscle Group | Hard Sets (7-day) | Volume Load (kg) | Status       | Suggestion                                                         |
|--------------|-------------------|------------------|--------------|--------------------------------------------------------------------|
| Quads        | 9                 | 4,050            | On target    | Maintain current volume                                            |
| Glutes       | 9                 | 4,500            | On target    | Maintain current volume                                            |
| Chest        | 9                 | 0 (BW)           | On target    | Progress to standard push-up if 12 reps achievable across all sets |
| Back         | 6                 | 0 (BW)           | Below MEV    | Add one set of inverted row next Full Body A session               |
| Shoulders    | 3                 | 0 (BW)           | Below MEV    | Add overhead push (pike push-up) to next Full Body B session       |
| Calves       | 6                 | 0 (BW)           | At MEV floor | Maintain; consider adding load if 20 reps achievable               |
| Core         | 9                 | 0 (BW)           | On target    | Maintain                                                           |

## 23. Definitions and Abbreviations

**CSS** — Cardio Stress Score — a derived metric combining cardio session duration and relative heart rate intensity, used to quantify the recovery cost of a cardio session on specific muscle groups.

**DOMS** — Delayed-Onset Muscle Soreness — muscle pain experienced 24–72 hours after unaccustomed exercise.

**Hard Set** — A working set completed with a  $RIR \leq 3$  or an  $RPE \geq 7$ ; used as the primary volume currency in the system.

**HRV** — Heart Rate Variability — a marker of parasympathetic nervous system activation used to assess recovery quality.

**MAV** — Maximum Adaptive Volume — the training volume at which adaptation is greatest; above this, fatigue accumulates faster than muscle is built.

**MEV** — Minimum Effective Volume — the lowest training volume that produces measurable adaptation for a given muscle group.

**MRV** — Maximum Recoverable Volume — the highest training volume the body can recover from between sessions.

**PNF** — Proprioceptive Neuromuscular Facilitation — an advanced stretching technique using isometric contractions to achieve greater passive range-of-motion gains than static stretching alone.

**RIR** — Reps in Reserve — the number of additional repetitions that could have been performed with good form at the end of a set.

**RPE** — Rate of Perceived Exertion — a subjective scale of effort intensity from 1 (very easy) to 10 (maximum effort).

**SMR** — Self-Myofascial Release — pressure applied to soft tissue (e.g. via foam rolling) to reduce tension and improve short-term range of motion.

**1RM** — One-Repetition Maximum — the maximum load that can be lifted for a single complete repetition with good form.



# Personal Workout App & Coaching System

## Implementation Improvements & Hardening Addendum

### Version 3.0 Addendum

| Field         | Detail                                                                                                              |
|---------------|---------------------------------------------------------------------------------------------------------------------|
| Document role | Supplement to the Version 2.0 Design & Technical Specification                                                      |
| Purpose       | Convert the existing concept into a safer, testable, implementation-ready product plan                              |
| Scope         | MVP definition, schema hardening, API contracts, safety, testing, observability, analytics, and acceptance criteria |
| Status        | Recommended implementation baseline                                                                                 |
| Date          | June 2026                                                                                                           |

**Recommended change-log entry:** Version 3.0 adds MVP prioritisation, normalized workout-template and session data, database constraints, migration procedures, versioned API contracts, deterministic coaching boundaries, readiness and safety checks, testing requirements, diagnostics, analytics, and measurable acceptance criteria.

# 24. Implementation Improvements and Product Hardening

The Version 2.0 specification establishes a strong local-first foundation. This addendum narrows the initial release, removes implementation ambiguity, and adds controls needed for reliable real-world use.

## 24.1 Recommended Release Scope

| Release      | Included capabilities                                                                                             |
|--------------|-------------------------------------------------------------------------------------------------------------------|
| MVP          | Profiles; exercise library; Full Body A/B templates; workout logging; weight, reps, RPE and RIR; workout history; |
| Version 1.1  | Notifications; cardio logging; scheduled and reactive deloads; improved settings.                                 |
| Version 1.2  | RPE alerts; readiness check; CSV import/export; goal-based planning; richer analytics.                            |
| Version 2.0+ | Ollama explanations; adaptive feedback; FIT/GPX import; wearable integration.                                     |

The MVP should prove that users can complete and review workouts reliably before adaptive AI is introduced. Advanced coaching features should remain behind feature flags until the deterministic rule engine has sufficient test coverage.

## 24.2 Data Model Normalization

Add explicit template and session-exercise layers. This prevents duplication, improves ordering, and cleanly separates planned work from completed work.

```
workout_templates(  
  template_id, name, description, template_type,  
  is_active, created_at  
)  
  
template_exercises(  
  template_exercise_id, template_id, exercise_id,  
  exercise_order, target_sets, rep_min, rep_max,  
  target_rpe, rest_seconds  
)  
  
session_exercises(  
  session_exercise_id, session_id, exercise_id,  
  exercise_order, planned_sets, notes  
)  
  
session_sets(  
  set_id, session_exercise_id, set_number,  
  reps_completed, weight_kg, rir, rpe,  
  is_warmup, completed_at  
)
```

Use one key strategy consistently. UUIDs are preferred when future synchronization or import merging is possible; integer identity keys are acceptable for a permanently local-only product.

# 25. Database Integrity and Migration Strategy

## 25.1 Required Constraints

```
CHECK (rpe BETWEEN 1 AND 10)
CHECK (rir BETWEEN 0 AND 10)
CHECK (reps_completed >= 0)
CHECK (weight_kg >= 0)
CHECK (duration_min > 0)
CHECK (rating IN (-1, 1))
UNIQUE (session_exercise_id, set_number)
```

Foreign keys should use explicit delete behavior. Deleting a workout session may cascade to its session exercises and sets, while deleting an exercise that appears in history should be blocked or converted into an archived exercise.

## 25.2 Schema Migrations

```
schema_migrations(
  version INTEGER PRIMARY KEY,
  name TEXT NOT NULL,
  applied_at TIMESTAMP NOT NULL,
  checksum TEXT NOT NULL
)
```

- Open the active profile database and read its schema version.
- Create a verified backup before destructive changes.
- Apply missing migrations in numeric order inside transactions.
- Record the checksum and completion timestamp.
- Rollback and stop startup safely if validation fails.
- Run the same migration set against every profile database.

## 25.3 Profile Storage and Safe Deletion

```
app_config.json
profiles/
  <profile-uuid>/
    profile.json
    workout.duckdb
  backups/
```

Use a UUID for the directory name and keep the display name inside profile.json. Profile actions should support archive, export, and permanent delete. Permanent deletion must require confirmation and should create a final export when possible.

## 25.4 Backup and Recovery

- Create a rotating backup before migrations and major imports.
- Use a temporary file plus atomic rename to avoid partial backups.
- Verify restored row counts, schema version, and database readability.
- Expose last-backup time and restore actions in Settings.
- Never overwrite the only valid backup during restore testing.

## 26. Versioned API Contracts

All endpoints should begin with /api/v1. Request and response bodies must be defined with Pydantic models, examples, status codes, and stable error structures.

### 26.1 Session Creation Example

POST /api/v1/sessions

```
{
  "sessionId": "2026-06-28",
  "templateId": "full-body-a",
  "exercises": [{
    "exerciseId": 4,
    "sets": [{
      "setNumber": 1,
      "reps": 10,
      "weightKg": 20,
      "rpe": 8,
      "rir": 2
    }]
  }]
}
```

201 Created

```
{
  "sessionId": "uuid",
  "status": "completed",
  "summary": {
    "hardSets": 12,
    "volumeLoadKg": 2480,
    "averageRpe": 7.8
  },
  "suggestionsGenerated": 2
}
```

### 26.2 Standard Error Behavior

| Status | Meaning                                                    |
|--------|------------------------------------------------------------|
| 400    | Malformed or logically invalid input                       |
| 404    | Profile, session, template, or exercise not found          |
| 409    | Duplicate profile, conflicting update, or duplicate import |
| 422    | Field-level validation failure                             |
| 500    | Unexpected database or migration failure                   |
| 503    | Optional Ollama service unavailable                        |

Session insertion must be atomic. The backend should either write the session, exercises, and all sets successfully or write none of them.

### 26.3 Idempotency and Imports

Imports and retryable writes should accept an idempotency key or source record identifier. Duplicate detection should combine source, external identifier, timestamp, and activity type rather than relying only on the session date.

# 27. Deterministic Coaching and Adaptive Feedback

## 27.1 Required Processing Boundary

```
Workout data
-> analytics engine
-> deterministic rule engine
-> structured recommendation
-> optional Ollama explanation
-> Flutter presentation
```

The LLM may explain recommendations, summarize trends, and answer questions using supplied facts. It must not silently change set counts, weights, thresholds, deload status, pain guidance, or medical meaning.

## 27.2 Structured Recommendation Contract

```
{
  "ruleKey": "reduce_leg_volume_high_cardio",
  "severity": "medium",
  "exerciseIds": [3, 7],
  "recommendedAction": {"setReduction": 2},
  "evidence": {"rollingCss": 214, "threshold": 200}
}
```

## 27.3 Safer Adaptation Rules

- Require at least 10-15 relevant feedback events before changing a rule.
- Limit a learned threshold adjustment to a small range, such as plus or minus 10 percent.
- Apply changes gradually and record the previous value.
- Allow the user to inspect and reset learned preferences.
- Never weaken pain, fatigue, or other safety rules based on thumbs-down feedback alone.
- Collect a dismissal reason such as too difficult, too easy, unavailable equipment, time limit, discomfort, inaccurate recommendation, or already completed.

```
suggestion_feedback(
  suggestion_id, session_id, rule_key, rating,
  dismiss_reason, accepted_action,
  context_snapshot, logged_at
)
```

## 28. Readiness, Fatigue, and Safety

### 28.1 Pre-Workout Readiness Check

```
sleep_quality 1-5
muscle_soreness 1-5
stress_level 1-5
energy_level 1-5
pain_present yes/no
```

Deload and fatigue decisions should combine multiple signals: rising RPE, falling repetitions, reduced estimated strength, excessive weekly volume, high cardio load, low readiness, missed sessions, and pain flags. A single noisy metric should usually create an advisory rather than a mandatory action.

### 28.2 Safety Boundaries

- The app is a training aid and not medical advice.
- Sharp pain, dizziness, chest pain, faintness, or unusual shortness of breath must trigger a stop message and appropriate professional-care guidance.
- Pain must not be classified as ordinary fatigue or soreness.
- The LLM must not diagnose injuries or override deterministic safety messaging.
- Onboarding should collect experience level, available equipment, excluded exercises, movement limitations, and preferred session duration.
- Health-related fields must remain optional, local, and excluded from routine logs.

### 28.3 Evidence Wording

Avoid absolute claims such as describing every cool-down as non-negotiable or stating that one duration of foam rolling is universally required. Use cautious wording and attach references for weekly volume, RPE/RIR, progressive overload, deloading, concurrent training, warm-ups, foam rolling, and estimated one-repetition maximum formulas.

# 29. Flutter Architecture and Workout Experience

## 29.1 Recommended Flutter Structure

```
lib/  
  app/  
  core/  
    api/  
    errors/  
    routing/  
    theme/  
    widgets/  
  features/  
    profiles/  
    exercises/  
    workouts/  
    analytics/  
    coach/  
    settings/  
  data/  
    models/  
    repositories/
```

Use one state-management approach consistently. A practical stack is Riverpod, go\_router, Dio, Freezed/json\_serializable, and a small local preference store for application configuration. Screens should depend on repositories rather than calling the API client directly.

## 29.2 Workout Logging Requirements

- Copy previous-session values into the current workout.
- Provide an optional rest timer and automatic set numbering.
- Support one-tap RPE/RIR entry and warm-up-set marking.
- Allow exercise substitution, reordering, skipping with a reason, and adding sets.
- Autosave an in-progress session and restore it after app restart.
- Warn before accidental exit when unsaved work exists.
- Show the previous performance and personal best beside each exercise.
- Update analytics immediately after an atomic session submission.

## 29.3 Analytics Dashboard

Include weekly hard sets by muscle group, training frequency, total duration, average RPE, planned versus completed sets, exercise progression, cardio duration/load, deload history, adherence, and personal records. Estimated 1RM should be labeled carefully for high-repetition and bodyweight movements.

# 30. Testing, Diagnostics, and Acceptance Criteria

## 30.1 Test Matrix

| Layer       | Minimum coverage                                                                                                         |
|-------------|--------------------------------------------------------------------------------------------------------------------------|
| Backend     | Transaction rollback; constraints; volume calculations; hard-set classification; RPE trends; deload scheduling; cardio   |
| Flutter     | Widget validation; navigation; state changes; profile switching; notifications; accessibility; interrupted-workout recov |
| Integration | Flutter -> FastAPI -> DuckDB; Flutter -> FastAPI -> Ollama; import -> validation -> database; backup -> restore -> r     |

Suggested targets: at least 90 percent coverage for deterministic coaching rules, 80 percent for backend services, and automated integration coverage for all critical workout flows.

## 30.2 Local Diagnostics

- Rotating application, backend, and import logs.
- A diagnostics screen showing backend, DuckDB, Ollama, schema version, active profile, last backup, last import, and app version.
- No default logging of complete workout notes, personal LLM prompts, raw imported health files, or full database rows.

## 30.3 Workout Logging Acceptance Criteria

- A user can start a saved template in no more than three taps.
- Previous set values appear automatically when history exists.
- Invalid RPE, RIR, repetition, and weight values are rejected before persistence.
- A completed workout is saved atomically.
- An unfinished workout survives application restart.
- Weekly analytics refresh immediately after successful submission.
- Backup restore produces the same schema version and expected row counts.

## 30.4 Recommended Implementation Order



| Priority | Work item                                                                      |
|----------|--------------------------------------------------------------------------------|
| 1        | Finalize MVP and acceptance criteria                                           |
| 2        | Normalize templates, session exercises, sets, and constraints                  |
| 3        | Implement migrations, transactions, backup, and recovery                       |
| 4        | Publish versioned API contracts                                                |
| 5        | Complete workout logging and history experience                                |
| 6        | Add deterministic analytics and coaching rules with tests                      |
| 7        | Add readiness, safety, notifications, and diagnostics                          |
| 8        | Introduce Ollama explanations and adaptive feedback only after core validation |